

Getting started

A guide for a hardware engineer who has never used this tool. It assumes you know what a flip-flop, a FIFO and a ready/valid handshake are, and that you have written Verilog. It assumes nothing at all about formal verification, answer-set programming, or the theory underneath — where those matter, this guide explains them in ordinary words at the moment they first matter.

Read this before the other documents. When you finish it you will know what each file in a block is for, what to type, what comes back, and what to do when something goes wrong.

1. What problem this solves

You are building a block. You know what it should do — you could explain it at a whiteboard in five minutes. Between that explanation and working silicon sit thousands of decisions, and the usual way to check them is a testbench: you think of situations, you write stimulus, you look at whether the outputs were right.

A testbench can only ever check the situations somebody thought of. The bugs that survive to silicon are the ones nobody thought of — the third simultaneous event, the entry that was reallocated in the same cycle its fill landed, the counter that was correct for eight cycles and wrong on the ninth.

This tool replaces “situations somebody thought of” with **all of them**. You state what the block promises in structured English; the tool turns that into a mathematical contract and then proves — over every reachable state and every possible input sequence, for all time — that your design cannot break it. When the proof fails it hands you back the exact situation that breaks it, which is usually more useful than the proof succeeding.

Two things it is not. It is not a replacement for you knowing your design: you still write the RTL logic (or a model writes it and you approve it). And it is not push-button: the one step no machine can check is whether the English you wrote is the block you meant, so the tool keeps a person in the loop at every stage on purpose.

2. The mental model: six artifacts and why each exists

A finished block is six files (plus the reports). This is the whole system; everything else in this documentation is detail about one of them.

#	artifact	who writes it	what it is
1	SPECIFICATION.md	you	the English in force: what the block promises, with every ambiguity decided and recorded
2	<block>.yaml	you	the <i>signature</i> : the wires, their widths, their protocols. The tool’s symbol table
3	<block>.cnl	you	the specification again, in <i>controlled English</i> — sentences with exactly one meaning each

#	artifact	who writes it	what it is
4	<code>spec.lp</code>	the tool	the contract : your sentences as mathematics. Generated; you read it, you never edit it
5	<code>l1.lp (+ l1.inv.lp)</code>	you , or a model you supervise	the <i>design</i> : your logic, written as facts the prover can reason about
6	<code><block>.sv</code>	the tool	the SystemVerilog, printed from the design and checked against a simulator

The shape to hold in your head:

```

you write ENGLISH  ->  the tool makes a CONTRACT  ->  you write a DESIGN
                        \                               /
                        \                               /
                        the CERTIFICATE proves one against the other
                          |
                          printed as RTL

```

Why the contract is generated rather than written. Writing the contract by hand is the specialised skill this tool exists to remove — it demands knowing tricks like wrap-bit pointers and knowing that a derived counter is secretly an enumeration that will bring the prover to its knees. You write sentences; the compiler knows the tricks.

Why the design is separate from the specification. If one program produced both the contract and the RTL, any misreading would appear identically on both sides and cancel out — the check would be comparing a thing with itself. Keeping them apart is what makes the certificate mean something.

Two commands worth knowing before you write anything. `sv2asp2 keywords` prints the controlled English’s vocabulary, marking the words that must carry an @. `sv2asp2 schema` prints the ASP you will have to write for the design and its linkage — every fact with its arity, every primitive with its pins, and the contract’s own vocabulary so you can read what the tool hands back. Both are printed by the tool that enforces them, so neither can describe something the tool would refuse.

`sv2asp2 schema --design` lists the facts a design may contain *and every operator* an expression may use, with its argument order — including `pack(L)`, which names a lane of bits as one word. When the tool is updated, `docs/spec2rtl2/CHANGES.yaml` is the one page that says what changed and why; its open entries are what the tool does not do yet.

The ladder. Each of those artifacts is a *rung*. The tool builds one, explains it, and then stops until you approve it. You approve by editing `ladder.yaml`; no command can do it for you, deliberately. If you are working with a coding agent, this is your steering wheel: it cannot run ahead of you.

3. Setting up

Full detail is in the suite book’s Part C; the short version:

```
cd tools          # the folder you were given
./setup.sh        # finds or installs Python 3.11+, builds .venv, checks the tools
```

It assumes nothing about your machine — no conda, no particular Python. It ends by running `doctor`, which tells you what is present and what to install. Two system tools matter: **clingo** (the prover) and **iverilog** (the simulator that arbitrates the final check).

Where you work. The tools folder is not where your block lives:

```
parent/
  tools/      the tool, the skill, the documentation, worked examples
  myBlock/    YOUR folder -- you run your session here, everything you make lives here
```

From `myBlock/` the tool is `../tools/.venv/bin/python -m sv2asp.aspfirst2 <verb>`, or run `../tools/.venv/bin/activate` once and then just `sv2asp2 <verb>`. This guide writes the short form.

4. Walkthrough: reading a real block

The tools folder ships a complete block at `examples/spec2rtl2/rv_missq/` — an instruction-fetch miss queue. In two sentences: when the instruction cache misses, the line has to be fetched from memory, and this block keeps track of the outstanding fetches. It holds a few *entries*, each standing for one cache line being fetched; a *demand* miss (the pipe is stalled waiting) outranks a *prefetch* (speculative), and a second request for a line already being fetched should join the existing entry rather than start a second fetch.

You do not need to understand the miss queue to follow this section. Look at the *shape* of each file — that shape is the same for every block, including yours.

4.1 The signature — `rvMissq.yaml`

This is the tool’s symbol table: every wire, its width, and how it behaves. Nothing else in the block names a wire, so without this file the specification would denote nothing.

`interfaces:`

- `name: "request"`
 `protocol: "readyValid"`
 `side: "receives"`
 `description: "A miss arriving from the fetch pipe, demand or prefetch"`

An *interface* is a bundle of wires that act together. `readyValid` means the familiar handshake: the sender asserts valid, the receiver asserts ready, and the transfer happens when both are high. `validOnly` means there is no back-pressure — the receiver must always take it.

Ports are listed individually, and one field on each of them matters more than all the others:

- `name: "requestAddress"`
 `interface: "request"`

```
role: "opaque"           # <- this
width: "addressWidth"
```

A port that is an **ARRAY** says so, with elements beside width (width is one element's width). It matters because the shape decides what a subscript means: on an array port `data[J]` is element J — an ordinary addressed read, compared as a whole token — and on a flat port it is bit J. Declare it and you can claim things per element; leave it out and every subscript is a bit.

role is the single most consequential thing you will write. `opaque` says: this is a *token* — the block moves it around and compares it for equality, but never reasons about its numeric value. `numeric` says: the value itself matters. Get this wrong on a 26-bit address and you ask the prover to consider 67 million values instead of one equality bit; the block will still be correct, but the proof will never finish. As a rule: addresses, tags and data are opaque; a mode bit or a small count is numeric.

4.2 The controlled English — `rvMissq.cnf`

This is where you say what the block does. It is English — you can read it aloud — but every sentence pattern has exactly one meaning, and a sentence outside the patterns is *refused by name* rather than guessed at. Two real declarations:

```
@behavior allocateDemand
  @when a valid demand request R arrives
  and roomForDemand
  and no joinable entry has R.address
  @then
    accept R
    create entry E for address R.address with tag R.tag
    @next cycle E is demanding and wantsFetch

@property neverOverfilled
  liveEntries @must @never exceed depth
```

Three things to notice, because they are the language's whole design:

1. **@behavior says what the machine DOES; @property says what must be TRUE of it.** Keeping those apart is what lets the tool check your specification against itself before any hardware exists. A promise smuggled into a behaviour would become true by construction, and the proof would be a tautology.
2. **The @ marks the words that give a sentence its shape** — `@when`, `@then`, `@must`, `@never`, `@next`. Ordinary words stay plain. You can see the skeleton at a glance, and a typo fails *as a keyword* instead of silently becoming prose.
3. **a, no, every are quantifiers, not decoration.** “a valid demand request R arrives” introduces R; “no joinable entry has R.address” is a real universal claim.

You never write a hold rule, and that is not a shortcut — it is the point. A behaviour says what happens when its event occurs. Read literally, that says nothing at all about the cycles when the event does *not* occur, which would leave the design free to change the state whenever nothing was happening. So the compiler adds a second monitor for every piece of state a behaviour writes: **a change with no cause that licensed it is a named failure.**

That is what pins a received byte to the wire it arrived on, without your writing a word about it — the bit

was captured, nothing else licensed a change, so it must still be there when it is presented. Writing the hold yourself is worse than unnecessary: a rule saying “if you hold a word, keep holding it” forbids a design that releases one word and accepts another on the same edge, which no requirement ever asked to forbid.

Two things worth knowing about it. The licence is specific: writing bit 3 licenses bit 3 and no other position. And only state a behaviour *writes* is framed — something your specification merely reads is a view of the design, not state you took responsibility for. In the generated contract these monitors are named `<window>Disturbed`; when you read it, check that the ones you expected are there.

Compiling this writes `rvMissq.cnf.core` beside it — the same specification in a symbolic notation. It is generated, committed, and never hand-edited; its value is that you can *diff* it to see exactly what your English came to mean.

4.3 The contract — `spec.lp`

```
sv2asp2 compile rvMissq.cnf rvMissq.yaml -o spec.lp
```

Out comes the contract: your promises as rules a prover can use. You read it; you never write it. A single line, so it is not mysterious:

```
failType(neverOverfilled, T) :- live(T), not neverOverfilledHolds(T).
```

Read it right to left: *at any instant T where the block is out of reset, if “neverOverfilled holds” is not true, then record a failure named neverOverfilled at T*. Every promise you wrote becomes one of these named failure conditions. The proof’s job is to show none of them can ever fire.

The compile also prints two things worth reading. **Refusals** name anything it would not lower (a boundary, stated honestly — not a crash). And the **mount manifest** lists the *windows* the contract needs — which brings us to the one genuinely unfamiliar idea.

4.4 The design, and the idea of a window — `l1.lp` and `l1.inv.lp`

The design is your logic, written as facts rather than as Verilog. A slot’s “valid” bit, for instance:

```
net_lane(validM1, depth, 1).
def_lane(validM1, I, logor(alloc(I), logand(valid(I), lnot(ends(I))))).
inst_lane(uValid, arff, depth).
```

That reads: there is a combinational net `validM1`, one per entry; its value is *allocated this cycle, OR (was valid AND is not ending)*; and it feeds a reset flip-flop whose output is `valid`. It is ordinary RTL thinking — next-state logic feeding a flop — in a notation the prover can read. (This is also the shop convention the printed RTL follows: every register `xx` is fed by a combinational `xxM1` carrying its complete next value.)

Now the important idea. Your specification talks about “entries” and whether one *exists*. Your design has a `valid` bit. Something must connect them, and the naive answer — have the specification keep its own list of entries — is a trap: that list is a second state machine, and you would then have to prove *it* consistent with the design in every state, which is harder than the original problem and is what makes naive formal attempts crawl.

Instead the specification declares **windows**: named views it needs, with no implementation. The design supplies them in one line each, in `l1.inv.lp`:

```
entryExists(0, T) :- val(valid(0), 1, T).
entryAddress(0, A, T) :- val(addr(0), A, T), val(valid(0), 1, T).
```

“Entry 0 exists at time T exactly when slot 0’s valid bit is 1.” No copy, no bookkeeping — a pane of glass over a register you already have. This is why proofs here finish in seconds rather than never.

4.5 The certificate — the actual proof

sv2asp2 certificate .

```
flow: verify.json
refine l1.lp: OK
refine l1.lp (induction only): OK
refine l1.lp: OK
second point (depth=2, maxOutstandingFetches=1): OK
second point discriminates: l1.lp rejected, as it must be
FLOW: OK
```

That is the proof, in four runs (§5 explains why four). Now read what the first one actually established — the report in `certificate.log`:

```
live: OK -- some instant is judged within the base window (the monitors can fire)
base: OK -- from reset, no property can fire on ANY input sequence within 1 live step(s)
induct K=1: state freed at T=0: askPtrHot(0)(1), ... ptrHot(3)(1), ...
induct: no ghost state (no history predicate in the monitors)
induct: reset-exempt monitor(s) NOT EXERCISED in this step (reset held released, so
  they can never fire here): ['emptyUnderReset', 'quietUnderReset']
induct: INDUCTIVE at K=1 -- [...]: with the base, they hold for ALL time
```

Line by line, because this is the output you will read most often:

- **live: OK** — before anything else, the tool checked that some instant can be judged at all. Every claim is silenced during reset; if nothing could ever be un-silenced (a block with no reset once compiled that way), every claim would pass for the wrong reason, so this line has to come first.
- **base: OK** — starting from reset, nothing can go wrong within the window, whatever the inputs do. This is the induction’s base case.
- **state freed at T=0** — the inductive step does *not* start from reset. It starts from an arbitrary state: every flip-flop set to anything at all. That is what makes the result cover all reachable states rather than a simulation’s worth of them.
- **no ghost state** — the contract watches the design through windows only. If you ever see ghost state reported, your specification is keeping a notebook, and it will be slow.
- **NOT EXERCISED** — honesty. Those two promises are about behaviour *during* reset, and this run holds reset released, so they could not possibly fire here. Rather than list them as proven, the tool excludes them and says so; the second run (with reset free) is where they are actually checked.
- **INDUCTIVE at K=1** — the payoff. “If the promises hold now, they still hold one cycle later” — which, with the base case, means they hold **forever**, for every input sequence.

4.6 The RTL, and the last check

```
sv2asp2 print l1.lp -o rvMissq.sv
sv2asp2 roundtrip l1.lp roundtrip_scenario.lp --icarus
```

icarus agrees on all 2679 definite samples

ROUNDTrip: OK

The print gives you parameterised SystemVerilog with generate blocks and ordinary always_ff staging — RTL a person can review. The round trip then closes the loop: the printed file is translated *back* by an independent translator and compared against the design cycle by cycle, with Icarus Verilog as a third opinion. This catches the one thing the proof cannot — a mistake in the printer itself.

5. Why the certificate is four runs

1. **The standard run** — the proof above.
2. **The strong half** (`--free-reset`) — the same induction with reset free at any instant, so the reset-time promises are actually exercised.
3. **A second opinion** — where a block has an independently written contract, both must agree. (Optional; the miss queue has one.)
4. **A second configuration** — the design regenerated at a different size, with the contract's constants matched to it. A block whose parameters are real in the header but baked into the logic passes at the default and fails here — and the run also checks that the second configuration *rejects* the default design, because a check that accepts everything checks nothing.

You do not run these by hand: `verify.json` in the block's folder lists them, and `certificate` executes it.

6. Starting your own block

In order, stopping for approval after each:

1. **SPECIFICATION.md** — the English. Do the boring part first: go through it hunting for sentences that could be read two ways, decide each one, and write the decision down. This is where the real bugs are caught, before any tool runs.
2. **<block>.yaml** — the signature. List the interfaces, then the ports. Set `role` carefully: `opaque` for addresses, tags and data; `numeric` only where the value's arithmetic matters. Parameter defaults must be **the configuration you actually build** — a contract compiled at depth 8 will happily blame a depth-4 design for being full.
3. **<block>.cnl** — the controlled English. Start with `@behavior` declarations (what happens when), then `@property` declarations (what must always be true), then `@scenario` declarations (situations that must remain *possible* — these catch the embarrassing failure where everything is provable because nothing can happen).
4. **compile**, then read the contract and the manifest.
5. **The design**, mounting every window the manifest named.
6. **certificate**, then print and roundtrip.

Copy `examples/spec2rtl2/rv_missq/` as your model for file layout, `verify.json` and `ladder.yaml`. The skill (`.claude/skills/spec2rtl-dsl/SKILL.md`) is the same procedure written as an operating manual, with every trap the route has paid for.

7. When it does not work

what you see	what it means	what to do
REFUSED: ... from compile	a sentence outside the frozen patterns, named exactly	rewrite that sentence in a pattern the language has; never work around it
NOT inductive at K=1	the promises are not strong enough to prove themselves one cycle at a time	read the counterexample table first — it is usually a real hole; then try a larger K; only then add a confining claim
a counterexample with impossible-looking state	the step starts from <i>any</i> state, including unreachable ones	if the state truly cannot occur, say why as a property — that is a real missing promise
scenario ... VIOLABLE	a situation you said must be possible can be defeated	usually the design; sometimes the scenario asks for something the block never promised
DARK READ / dangling atoms in the round trip	the printed RTL reads something nothing drives	a translator-level gap: report it (§8)
the step never finishes	something is being enumerated — almost always a wide value marked numeric	make it opaque, or model it as a token

The rule behind the table: **a report’s exclusion lines are part of its verdict.** “reset held released”, “NOT EXERCISED”, “bounded-only” each narrow what was proven, and reading them is how you know what you actually have.

8. Working with a coding agent, and getting help

Start your session in your block’s folder and point it at `../tools/` — the `CLAUDE.md` there tells it the layout, the rules it must not break, and where the skill and the worked example are. What keeps this safe is the ladder: the agent builds one artifact, explains it, and stops. You read the explanation and approve it before anything else happens.

If the tool refuses something it should accept, add `--report issue.txt` to the command and send that file to whoever maintains your copy. It contains the tool version, the toolchain it resolved, the command and the output — and nothing from your design. Do not patch the installed tool: a certificate from a modified tool means nothing.

9. Where to read next

you want	read
every command, and what is automated	<code>AUTOMATION.md</code>
the language reference — every sentence pattern	<code>ROUTE_METHODOLOGY.md</code> , Part II
what the tool guarantees, and its architecture	<code>SUITE.md</code> , Parts A and B
the same route walked on a smaller block	<code>WALKTHROUGH.md</code>
the operating procedure, with every trap	<code>.claude/skills/spec2rtl-dsl/SKILL.md</code>